

Pseudo code for Genetic Algorithm for optimization problem for in Python language.

Define Objective-function (x)

Set population_size

Set number_of_generations

Set mutation_rate

Set crossover_rate

population = Generate random individuals (population_size)

for generation = 1 to number_of_generations:

for each individual in population fitness =
objective-function(individual)

Parents = select individuals based on fitness

new_population = []

While size of new_population < population_size

parent 1, parent 2 = select two parents

If random() < crossover_rate:

child 1, child 2 = cross over (parent 1, parent 2)

ELSE:

child 1 = parent 1

child 2 = parent 2

If random() < mutation_rate:

child 1 = mutate(child 1)



child 2 = mutate (child 2):
Add child 1 and child 2 to new-population

population = new-population

best = Best Individual in population

Print (generation and best fitness).

Return best

END

TSP

Title

Problem Objective

Code summary.

Output summary

Title of the Paper:

"Travelling Salesman Problem Optimization Using Genetic Algorithm"

Published in "2019"

Problem Statement:

The Travelling Salesman Problem (TSP) requires finding the shortest distance for the salesman to visit a given set of cities exactly once and return to the starting city. The Genetic Algorithm (GA) is used to find an optimized route by applying

Selection Crossover & mutation operations to differ-
-nt possible routes

Code Summary

- (1) The Program import the random module to generate random routes & perform mutations
- 2) A distance matrix is created to store the distance between every pair of cities.
- 3) The number of cities, population size, generations & mutation rate are defined.
- 4) The calculate distance function calculates the total distance of a complete route.
- 5) The program creates an initial population containing randomly generated routes
- 6) the selection() function selects the best routes having the shortest distance.
- 7) The Crossover() function combines two parent routes to produce a new valid route.
- 8) The Genetic algorithm repeats Selection, crossover & mutation for a fixed number of generations
- 9) After all generations the program identifies the route with minimum total distance
- 10) The best route is displayed
- 11) Finally, the program displays the optimized route & its minimum travel distance

Output Summary

- 1) The program displays the best / optimized route for visiting all cities
- 2) It displays the minimum total distance of the route.
- 3) The output shows that genetic Algorithm can find an efficient solution to the ~~TSP~~ TSP.

completed

cos
5/10/26